

## 4교시: 이미지 배포 흐름

### 수업 목표

- build, tag, push, pull, run 흐름을 배포 파이프라인의 축소판으로 설명한다.
- tag와 registry가 handoff와 rollback에 왜 중요한지 설명한다.
- public push 없이도 local evidence로 배포 흐름을 학습한다.

### 50분 흐름

| 시간     | 활동                         | 비중     | 산출            |
|--------|----------------------------|--------|---------------|
| 0-8분   | image lifecycle 복습         | 설명 15% | lifecycle map |
| 8-20분  | build-tag-push-pull-run 흐름 | 설명 25% | flow note     |
| 20-32분 | local build/tag 실습         | 실행 25% | tag evidence  |
| 32-42분 | registry push gate         | 설명 20% | push decision |
| 42-50분 | README 배포 흐름 작성            | 실행 15% | deploy note   |

### Visual 1: image 배포 흐름



이 visual은 source에서 image build, tag, registry, pull, run으로 이어지는 흐름을 보여준다.

## 핵심 설명

Docker image는 배포 가능한 artifact다. local에서 build한 image를 tag로 식별하고, registry에 push하면 다른 환경에서 pull해 실행할 수 있다. Day 5에서는 public push를 기본 요구하지 않지만, 흐름은 이해해야 한다.

중요한 것은 tag다. tag는 사람이 image를 식별하는 이름이다. latest만 쓰면 어느 시점의 image인지 설명하기 어렵다. 수업 evidence에는 :local, :v1, 날짜 또는 과제명 tag처럼 의미 있는 tag를 둔다.

## 흐름

```
source + Dockerfile
-> docker build
-> local image tag
-> optional registry push
-> pull from another host
-> docker run / compose up
-> verify evidence
```

## 실습 명령

```
cd week2/day5/labs/integration-app
docker build -t paperclip/week2-day5-integration:local .
docker tag paperclip/week2-day5-integration:local paperclip/week2-day5-integration:v1
docker images paperclip/week2-day5-integration
```

## tag 판단표

| tag        | 장점       | 한계             |
|------------|----------|----------------|
| latest     | 입력이 짧음   | 재현성 낮음         |
| local      | 실습용 구분   | 공유/배포 의미 약함    |
| v1         | 제출 기준 명확 | 변경 이력 관리 필요    |
| 날짜 tag     | 시점 기록    | 의미 없는 날짜 남발 가능 |
| commit sha | 정확한 추적   | 초급자에게 길고 복잡    |

## registry push 주의

registry push는 공유 행위다. image 안에 secret이나 사적 파일이 있으면 공개될 수 있다. Day 5 기본 실습은 push 없이 local tag와 run evidence로 충분하다.

push가 필요한 경우:

```
docker tag local-image USER/REPO:TAG
docker push USER/REPO:TAG
```

이 명령은 강사가 명시적으로 요청할 때만 사용한다.

## 학술 기준 연결

Twelve-Factor App의 build/release/run 분리는 Day 5 image flow와 잘 맞는다. Docker build는 build artifact를 만들고, tag는 release 식별자 역할을 하며, run은 runtime 단계다.

## 실무 failure mode

| Failure mode                  | 증상                  | 예방                 |
|-------------------------------|---------------------|--------------------|
| tag 없음                        | 어떤 image인지 모름       | explicit tag       |
| wrong repo tag                | push 실패 또는 엉뚱한 repo | tag 확인             |
| secret 포함 image push          | credential leak     | security gate      |
| local only image를 다른 PC에서 run | image not found     | registry/pull 필요   |
| build와 run 혼동                 | Dockerfile만 전달      | image/README 함께 제공 |

## 오해 점검

| 오해                             | 교정                             |
|--------------------------------|--------------------------------|
| build하면 자동 배포된다                | registry push/pull/run은 별도 단계다 |
| tag는 장식이다                      | artifact 식별과 handoff 기준이다      |
| push하지 않으면 학습이 안 된다            | local tag flow로도 원리를 학습할 수 있다  |
| Docker Hub 로그인 정보는 README에 적는다 | 절대 적지 않는다                      |

## 기록 템플릿

```
## Image Flow Evidence
- Source path:
```

- Build command:
- Local tag:
- Release-style tag:
- Push decision:
- Pull/run assumption:
- Verification:

## 평가 기준

| 기준      | 2점 evidence                      |
|---------|----------------------------------|
| 흐름      | build/tag/push/pull/run 순서를 설명했다 |
| tag     | latest만으로 부족한 이유를 설명했다           |
| 보안      | push 전 secret gate를 작성했다         |
| handoff | README에 image 실행 기준을 남겼다         |

## 전이 과제

Week 3에서 frontend와 api image가 따로 생긴다면 tag 전략은 어떻게 할지 쓴다.

frontend image tag:  
api image tag:  
둘이 같은 release에 속한다는 것을 어떻게 기록할 것인가:

## 공식 근거 링크

- Docker Hub repositories: <https://docs.docker.com/docker-hub/repos/>
- Twelve-Factor App: <https://12factor.net/>

## digest와 tag의 차이

초급 단계에서는 tag를 주로 사용하지만, 실무에서는 digest도 중요하다. tag는 사람이 읽기 쉬운 이름이고 digest는 content-addressed identifier다. 같은 tag가 시간이 지나 다른 image를 가리킬 수 있기 때문에 높은 재현성이 필요한 환경에서는 digest 또는 build provenance를 함께 본다.

| 식별자              | 장점                 | 한계                    |
|------------------|--------------------|-----------------------|
| tag              | 읽기 쉽고 운영자가 이해하기 쉬움 | mutable 가능            |
| digest           | content를 정확히 가리킴   | 사람이 읽기 어려움            |
| commit sha tag   | source와 연결 쉬움      | 초급자에게 복잡              |
| semantic version | release 의미 표현      | version discipline 필요 |

## push decision record

```
## Push Decision
- Do we need public sharing:
- Image checked for secrets:
- Tag:
- README ready:
- Decision: push / do not push
- Owner:
```

## 배포 evidence checklist

- [ ] source path가 명확하다.
- [ ] build 명령이 기록되어 있다.
- [ ] local tag가 있다.
- [ ] release-style tag가 있다.
- [ ] push 여부가 명시되어 있다.
- [ ] push하지 않았다면 이유가 있다.
- [ ] run/check evidence가 있다.
- [ ] cleanup 또는 rollback note가 있다.

## registry 없이도 학습할 수 있는 것

| 개념           | local 실습 방식          |
|--------------|----------------------|
| artifact 생성  | docker build         |
| artifact 식별  | docker tag           |
| runtime 검증   | docker run           |
| handoff      | README tag 기록        |
| release gate | push decision record |

## Lesson 4 Exit Ticket

```
## Exit Ticket
- 내 image local tag:
- release-style tag:
- push 여부:
- push하지 않는 이유:
- 다른 사람이 실행하려면 추가로 필요한 것:
```

## digest와 tag의 차이

초급 단계에서는 tag를 주로 사용하지만, 실무에서는 digest도 중요하다. tag는 사람이 읽기 쉬운 이름이고 digest는 content-addressed identifier다. 같은 tag가 시간이 지나 다른 image를 가리킬 수 있기 때문에 높은 재현성이 필요한 환경에서는 digest 또는 build provenance를 함께 본다.

| 식별자              | 장점                 | 한계                    |
|------------------|--------------------|-----------------------|
| tag              | 읽기 쉽고 운영자가 이해하기 쉬움 | mutable 가능            |
| digest           | content를 정확히 가리킴   | 사람이 읽기 어려움            |
| commit sha tag   | source와 연결 쉬움      | 초급자에게 복잡              |
| semantic version | release 의미 표현      | version discipline 필요 |

## release note 예시

```
## Image Release Note
- Image: paperclip/week2-day5-integration:v1
- Source path: week2/day5/labs/integration-app
- Build command:
- Verification:
- Push:
- Risk:
```

## pull/run 사고실험

다른 학생이 내 image를 pull해서 실행한다고 가정한다.

| 정보             | 이유            |
|----------------|---------------|
| repository/tag | 어떤 image를 받을지 |
| port           | 어디로 접속할지      |
| env            | 어떤 설정이 필요한지   |
| volume         | data가 남는지     |
| check          | 정상 상태 판단      |
| cleanup        | resource 정리   |

## 배포 흐름과 DORA preview

Day 5는 DORA 지표를 깊게 다루지 않지만, image flow는 delivery performance와 연결된다. build와 tag가 명확하면 change lead time과 rollback 판단이 쉬워진다. 반대로 artifact가 불명확하면 change failure 분석도 어려워진다.

## push decision record

```
## Push Decision
- Do we need public sharing:
- Image checked for secrets:
- Tag:
- README ready:
- Decision: push / do not push
- Owner:
```